Assignment_3

TeamID: 19
Dipesh Mishra - 20251651036
Suraj Kr Saw- 20251651092
Sandeep Parmar- 20251651081
Vivek Kumar- 20251651104

Task 1. Pick the partner edge and justify SOAP. Choose one external partner CampusEats must integrate — a payment gateway, a logistics/delivery partner, or an SMS gateway. In 3–4 sentences, explain why that edge is SOAP while the rest of CampusEats is REST (strong contract, message-level security, transactions).

Submit — a short "Context" section in integration.pdf naming the partner and the one operation you'll integrate.

For CampusEats we are choosing Payment Gateway. This is implemented using SOAP rather than REST as payment processing requires a strong contract, message level security, and transaction guarantee. All the payment system (like: SWIFT) are using SOAP, the back end software running major global banks relies heavily on SOAP, their existing system are highly secure.

SOAP ensures that sensitive information such as card details and authentication tokens are exchanged in a structured, secure, and verifiable manner. While the rest of CampusEats services (menu, orders, delivery, tracking) use REST for lightweight communication, the payment gateway integration demands SOAP to guarantee reliability and compliance with financial standards.

The single operation we integrate is **chargePayment**, which securely processes a user's payment for an order.

Task 4: Show the HTTP binding. Show the HTTP POST that carries the request: method, Host, Content-Type, the SOAPAction value from your binding, and the endpoint URL from your service/port.
Submit — the HTTP request block in integration.pdf.

```
POST /pay HTTP/1.1
Host: api.unipay.example
Content-Type: text/xml; charset=utf-8
SOAPAction: "urn:unipay:payments/charge"

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
               xmlns:uni="urn:unipay:payments">
  <soap:Header>
    <uni:auth token="CE-MERCHANT-KEY"/>
  </soap:Header>
  <soap:Body>
    <uni:charge>
      <cardToken>tok9f2c</cardToken>
      <amount>300</amount>
      <currency>INR</currency>
    </uni:charge>
  </soap:Body>
</soap:Envelope>
```

Task 5: Describe discovery (modern form). Explain how CampusEats would obtain this WSDL — a direct URL, or a lookup — and write a one-record catalogue / registry entry: business, service, endpoint, and the WSDL it speaks (a

Assignment_3

TeamID: 19
Dipesh Mishra - 20251651036
Suraj Kr Saw- 20251651092
Sandeep Parmar- 20251651081
Vivek Kumar- 20251651104

tModel-style pointer). No live UDDI server is required.
Submit — a "Discovery" section with the registry entry in integration.pdf.

Discovery Section:

CampusEats obtains UniPay's WSDL either via a direct URL (e.g., https://api.unipay.example/Payments.wsdl) or through a registry lookup. In practice, banks often email integrators the WSDL link directly, while catalogues provide a structured entry for discovery.

```
business: UniPay Bank Ltd.
service: Payments
endpoint: https://api.unipay.example/pay
spec (tModel): speaks urn:unipay Payments WSDL
```

Task 6: Map the fault into your own contract. Show how the partner's fault becomes the error your Assignment 2 contract promised (e.g. the partner's card_declined → your placeOrder "payment declined"). The partner's vocabulary must not leak to your students.
Submit — a short "Fault mapping" note in integration.pdf.

UniPay Fault:

```
<soap:Fault>
  <faultcode>soap:Client</faultcode>
  <faultstring>Card declined</faultstring>
  <detail>
    <uni:error code="card_declined"/>
  </detail>
</soap:Fault>
```

CampusEats mapping:

Reasoning:

The partner's code (card_declined) is machine readable for integration, but CampusEats exposes only its own error vocabulary to students. This ensures consistency with the Assignment 2 contract and prevents external system details from leaking into the user experience.